

Comparative Analysis of Interpolation Methods for 7-DoF Manipulator Robot Trajectory Planning

Vijay Bhaskar Senwal

Department of CSE
MANIT

Bhopal, India

vsemwal@manit.ac.in

Gera Ajith*

Department of Mechanical Engineering
MANIT

Bhopal, India

ajithgera63@gmail.com

Srinivasulu Vardhineni

Department of Mechanical Engineering
MANIT

Bhopal, India

srinivasulu.vardhineni@gmail.com

Yadunath Pathak

Department of CSE
MANIT

Bhopal, India

yadunath@manit.ac.in

Neha Gaud

Department of CS&IT
DAVV

Indore, M.P., India

neha28gaud@gmail.com

Abstract—This paper presents a comprehensive analysis of various interpolation techniques for trajectory planning of a 7-DoF Panda Robot under Robot Operating System(ROS2) environment. Total six distinct interpolation techniques, namely linear, circular arc, quadratic, cubic, quintic polynomial, and Bezier curves are implemented and analyzed. Various trajectories are evaluated based on root mean square error(RMSE) of empirical estimation of waypoints between estimated and true joint angles. The implementation relies on ROS2 environment to achieve real time trajectory interpolation and use error metrics offered by ROS2 for quantifying the performance of the trajectory interpolation process of each of the proposed methods. By reducing the number of waypoints and comparing the resulting joint angle accuracies, we set up strong correlations between waypoint density and trajectory precision for each interpolation method. The results provide valuable outcomes for choosing suitable methods and evaluating the corresponding errors to meet the requirement and Robot control for robotic applications in terms of accuracy, computational cost, and smoothness of trajectories. This work advances the development of heuristic, optimal and efficient motion planning algorithms for Industrial robotic systems especially for applications that require high accuracy of trajectory tracking.

Index Terms—Internet of Things (IoT), Robot Operating System (ROS), Robot, Motion Planning, Trajectory Control, Manipulator Robot.

I. INTRODUCTION

High-DOF robotic manipulators are increasingly being used widely by the industry and healthcare in manufacturing sectors to provide better flexibility. The motion planning is

very important in the successful completion of accurate and efficient manipulations in such systems. One of the biggest obstacles is to search the high-dimensional space for optimal DoF configurations, which is especially relevant given the fact that the present study involved the 7-DoF Franka Emika Panda robot.

Some motions such as those of the 7-DoF Panda robot based on redundant systems make the task of finding an optimal solution to the planning space quite a challenge, especially with tasks that need to be accomplished efficiently by a high quality and smooth trajectory [1]. In order to overcome this challenge, several path planning algorithms have been proposed. This approach is to use the Open Motion Planning Library (OMPL) embedded in the MoveIt! to address the high-level spatial algorithms and planner for the better robot model configuration and optimum motion planning [3].

There are also studies in recent years that focused on trajectory planning methods, where both position and orientation of an object or body are considered through cubic spline interpolation, resulting to higher order continuity and lesser fluctuations [3]. Moreover, Bézier curves have also been implemented successfully in spray painting robots, which is undertaking path planning and achieving smooth motion for curved surface [4]. For the purpose of the implementation and testing of these plans for the trajectory planning, the software frameworks like ROS2 and MoveIt2 are crucial. Using these frameworks, researchers can overcome many obstacles

encountered in analytics, like low code reuse, poor scalability, and trajectory planning.

In the recent past, research was conducted in order to determine motion trajectory planning of industrial robots using ROS, where MoveIt! and the OMPL library that are used in spatial linear and arc motion trajectory planning. The paper employed quintic spline interpolation for commanding smooth and accurate joint space trajectory and simulations and experiments demonstrated the efficacy of the system for attaining smooth and stable end-effector motion. This underlines a possibility of using ROS for the development of complex robotics applications for industrial facilities [5].

Besides, ROS2-based frameworks have been extended to many fields developing a unified framework that combines ROS2 and OpenPLC for efficient control of dextrous automation equipment [6]. In a similar manner, the Robot Operating System 2 (ROS 2) combined with the Navigation 2 (Nav 2) stack enabled autonomous navigation for the TurtleBot3 robot across simulation and actual testing [7]. Real-world deployed autonomous systems involved the creation of rover through ROS2 and deep learning models, that could work alongside the firefighters, to help obtain key information by navigating in the dangerous conditions and showed how ROS2 can help raise safety and effectiveness levels [8].

Further, a solution for urgent and generically configurable simulations of mobile manipulation tasks with ROS and MoveIt was presented, to improve system integration and visualization for applications in automatic manufacturing systems [9]. Combined, these investigations underscore the ROS based frameworks generalizability and the variety of problems they can solve in industrial automation, robotic motion planning and decision, and emergency response scenarios.

This work particularly assesses the performance of the trajectory planning algorithms in terms of error rate and computation time and test the interpolation methods, on the 7-DoF Panda robot. It is important to establish the children's starting point in terms of speed, effectiveness in executing a move or the quality of their motions. We compare the performance of six interpolation methods:

- 1) Linear interpolation - simpler and faster
- 2) Circular arc interpolation - smooth curved paths using circles
- 3) Cubic interpolation - smooth paths using third-degree polynomial
- 4) Quintic interpolation - extra smooth paths using fifth-

degree polynomial

- 5) Cubic-Spline interpolation - smoothly connected curved segments
- 6) Bezier curve interpolation - flexible curves using control points

We evaluate accuracy in joint angle and motion smoothness by three aspects:

- 1) RMSE analysis for individual interpolations: Assess the RMSE performance among Linear, Cubic, Quintic, Cubic-Spline, and Bezier curve interpolations for 5, 10 and 15 waypoints and group them from 5 to 15 comparing the RMSE values of each method
- 2) Joint angle accuracy comparison: Comparing the actual and calculated joint angles of Linear, Cubic, Quintic, Cubic-Spline, and Bezier curve interpolations.
- 3) Interpolation comparison at different waypoints: Evaluating the error in the visualization of using Linear, Cubic, Quintic, Cubic-Spline, and Bezier curve interpolations at different waypoints.

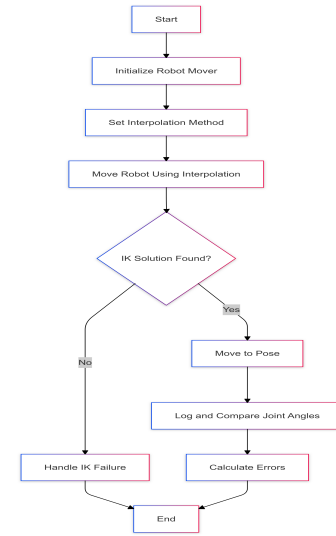


Fig. 1 Control Flow For Trajectory Planning With ROS2

II. ROS2 AND MOVEIT2 SYSTEM ARCHITECTURE

A. MoveIt2

MoveIt2 is utilized in the ROS2 software environment and allows even flexible robot control and further development as it offers a set of necessary tools. At the heart of it is Moving Average Group node that is the control interface, and providing command line in C++/Python realizations and GUI for visualization. MoveIt2 works in the robot architecture and interacts with all its URDF design files and SRDF capability

TABLE I
SUMMARY OF RELATED WORK

Author (Year)	Title	Description
Sotirchos & Ajanovic (2024)	Search-based versus Sampling-based Robot Motion Planning	Comparative analysis of motion planning approaches for 7-DoF Panda robot, focusing on optimal solutions for complex trajectory planning in redundant systems.
Yuan & Liu (2024)	An efficient RRT cache method in dynamic environments for path planning	Presents advancements in RRT-based path planning methods for dynamic environments, utilizing OMPL within MoveIt! for improved robot configuration and motion planning.
Liu et al. (2023)	Trajectory Planning of Robot End-Effector Position and Orientation Based on Cubic Spline	Investigates trajectory planning methods considering both position and orientation through cubic spline interpolation, achieving higher order continuity and reduced fluctuations.
Chen et al. (2017)	Path planning scheme for spray painting robot with Bézier curves	Implementation of Bézier curves in spray painting robots for smooth motion planning on curved surfaces, demonstrating practical applications in industrial settings.
Xu et al. (2022)	Research on Motion Trajectory Planning of Industrial Robot Based on ROS	Explores industrial robot trajectory planning using ROS, MoveIt!, and OMPL, employing quintic spline interpolation for smooth joint space trajectories.
He et al. (2022)	A ROS2-Based Framework for Industrial Automation Systems	Development of a unified framework combining ROS2 and OpenPLC for efficient control of industrial automation equipment.
Daubaris et al. (2024)	ROS2 as an Enabler for a Business Ecosystem	Case study on ROS2 and Navigation 2 stack implementation for autonomous navigation in TurtleBot3, covering both simulation and actual testing.
Rafiq et al. (2024)	Design of Autonomous Rover for Firefighter Rescue	Integration of ROS2 with deep learning for autonomous rover development in firefighting applications, demonstrating real-world safety applications.

files allowing it to effectively perform motion planning and navigation, sense 3D space, control its kinematics, and employ complex motion algorithms. The Move Group node can give real-time sensory feedback to the robot's controller, interact with controllers, and develop the best efficient motion path while the robot is in operation which makes robot development easier, faster and efficient.

B. ROS2

The Robot Operating System 2 (ROS2) is a powerful and modular Linux-based software infrastructure for maintaining robotic applications. The MoveIt! Subtool named Assistant

integrated in ROS2 allows users controlling virtual robot in efficient way due to the convenience of the interface. By organizing robot model, planning and visualizing motion planning, the MoveIt! Assistant allows users to control robots to perform various tasks thus making overall development more efficient.

III. TRAJECTORY PLANNING METHODS

The Equations for each of the interpolation are procedure used in the trajectory planning system introduced in this paper. For any approach, we define a start pose A_0 and end pose A_f ,

where where t is the interpolation parameter running between 0 and 1 $t \in [0, 1]$

A. Linear Trajectory Generation

For poses A_s and A_f with positions (u_s, v_s, w_s) and (u_f, v_f, w_f) :

$$q(\lambda) = q_s + \lambda(q_f - q_s) \quad (1)$$

where:

$$u(\lambda) = u_s + \lambda(u_f - u_s) \quad (2)$$

$$v(\lambda) = v_s + \lambda(v_f - v_s) \quad (3)$$

$$w(\lambda) = w_s + \lambda(w_f - w_s) \quad (4)$$

with $\lambda = \frac{k}{M-1}$ for $k = 0, 1, \dots, M-1$, where M represents waypoint count.

B. Circular Arc Trajectory Generation

For circular interpolation:

Arc center:

$$h = \frac{q_s + q_f}{2} \quad (5)$$

Plane normal:

$$m = \frac{(q_s - h) \times (q_f - h)}{|(q_s - h) \times (q_f - h)|} \quad (6)$$

Arc path for parameter λ :

$$q(\lambda) = h + T(\lambda\pi)(q_s - h) \quad (7)$$

where $T(\beta)$ represents rotation about axis m by angle β :

$$T(\beta) = \cos(\beta)I + (1 - \cos(\beta))mm^T + \sin(\beta)[m]_{\times} \quad (8)$$

C. Cubic Polynomial Interpolation

The cubic trajectory:

$$q(\lambda) = b_0 + b_1\lambda + b_2\lambda^2 + b_3\lambda^3 \quad (9)$$

with coefficients:

$$b_0 = q_s \quad (10)$$

$$b_1 = 0 \text{ (zero initial velocity)} \quad (11)$$

$$b_2 = 3(q_f - q_s) \quad (12)$$

$$b_3 = -2(q_f - q_s) \quad (13)$$

yielding:

$$q(\lambda) = q_s + 3(q_f - q_s)\lambda^2 - 2(q_f - q_s)\lambda^3 \quad (14)$$

D. Quintic Polynomial Interpolation

Quintic polynomial with continuous acceleration:

$$q(\lambda) = b_0 + b_1\lambda + b_2\lambda^2 + b_3\lambda^3 + b_4\lambda^4 + b_5\lambda^5 \quad (15)$$

where:

$$b_0 = q_s \quad (16)$$

$$b_1 = 0 \text{ (zero initial velocity)} \quad (17)$$

$$b_2 = 0 \text{ (zero initial acceleration)} \quad (18)$$

$$b_3 = 10(q_f - q_s) \quad (19)$$

$$b_4 = -15(q_f - q_s) \quad (20)$$

$$b_5 = 6(q_f - q_s) \quad (21)$$

E. Cubic Spline Interpolation

For waypoints $q_0, q_1, \dots, q_m$, cubic spline $K(\lambda)$:

$$K(\lambda) = e_i + f_i(\lambda - \lambda_i) + g_i(\lambda - \lambda_i)^2 + h_i(\lambda - \lambda_i)^3 \quad (22)$$

for $\lambda \in [\lambda_i, \lambda_{i+1}]$, coefficients e_i, f_i, g_i, h_i ensure:

- 1) $K(\lambda_i) = q_i$ for all i
- 2) $K'(\lambda_i^-) = K'(\lambda_i^+)$ for internal points
- 3) $K''(\lambda_i^-) = K''(\lambda_i^+)$ for internal points
- 4) Natural spline: $K''(\lambda_0) = K''(\lambda_m) = 0$

F. Bezier Curve Interpolation

For $m+1$ control points $Q_0, Q_1, \dots, Q_m$:

$$C(\lambda) = \sum_{i=0}^m Q_i D_{i,m}(\lambda) \quad (23)$$

where $D_{i,m}(\lambda)$ are Bernstein polynomials:

$$D_{i,m}(\lambda) = \binom{m}{i} \lambda^i (1 - \lambda)^{m-i} \quad (24)$$

For implementation with 4 control points:

$$C(\lambda) = (1 - \lambda)^3 Q_0 + 3\lambda(1 - \lambda)^2 Q_1 + 3\lambda^2(1 - \lambda) Q_2 + \lambda^3 Q_3 \quad (25)$$

G. Error Calculation

Root Mean Square Error (RMSE):

$$RMSE = \sqrt{\frac{1}{m} \sum_{i=1}^m (\phi_i^r - \phi_i^e)^2} \quad (26)$$

where:

- ϕ_i^r = reference joint angles

- ϕ_i^e = estimated joint angles
- m = waypoint count

IV. EXPERIMENTAL SETUP AND SIMULATION

As illustrated earlier in Figure 3, we first started the robot in the new MoveIt2 environment for our experiment. We then compared different interpolation techniques to the 7-DOF Panda robot to get an idea of how each worked and how they could be used. This allowed for the identification of clusters of respondents' profiles and correlation of the results with different variables in order to get a multi aspect overview of each of the methods under study. Another node was performed to set the movement parameters in terms of numbers of steps of the robot's path. For the simulation, the corresponding number of steps created waypoints, one more than the steps, to facilitate the interpolation as illustrated in the figure. The robot starting position to the end position was accomplished and the joint angles measured from the actual movement were compared to the theoretical joint angles at each waypoint. Last, an overall Root Mean Square Error (RMSE) was calculated to measure the efficacy of the movement.

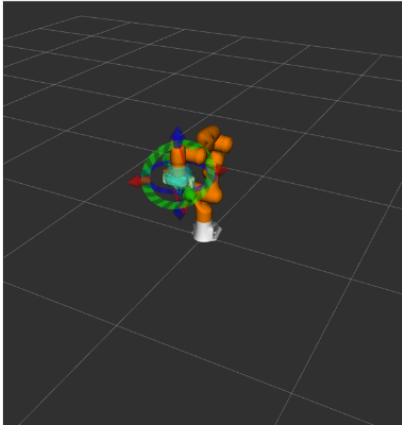


Fig. 2 Panda Robot Launched In RViz and MoveIt2 Environment

V. SIMULATION RESULTS AND DISCUSSION

A. Performance Analysis

The comparison of different interpolation methods revealed distinct performance patterns:

- B zier Curve Interpolation showed the best overall performance with the lowest RMSE values (1.720, 1.772, and 1.792 for 5, 10, and 15 waypoints respectively).

- Cubic Interpolation showed good performance with fewer waypoints (RMSE = 1.892 for 5 waypoints) but increased to 2.114 for 15 waypoints.
- Linear Interpolation showed a slight increasing trend in RMSE values (2.018, 2.114, 2.100) across different waypoint configurations.
- Circular Arc Interpolation demonstrated the highest RMSE values overall, with significant improvement as waypoints increased (2.668 to 2.155).

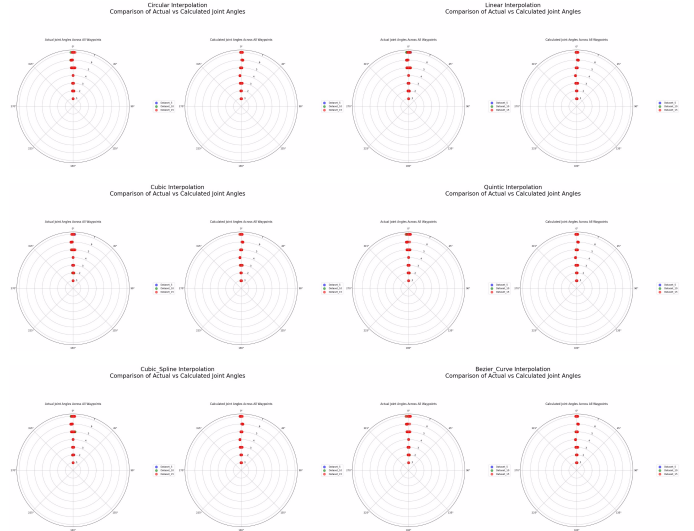


Fig. 3. Actual Angles vs. Calculated Angles

B. Impact of Waypoint Density

Analysis of waypoint density impact revealed two distinct patterns:

- 1) Methods showing improvement with increased waypoints:
 - Circular Arc Interpolation (19.2% improvement from 2.668 to 2.155)
 - Quintic Interpolation (1.9% improvement from 2.143 to 2.184)
- 2) Methods showing performance degradation:
 - Cubic Spline (15.5% degradation from 2.026 to 2.325)
 - B zier Curves (4.2% degradation from 1.720 to 1.792)
 - Linear Interpolation (4.1% degradation from 2.018 to 2.100)

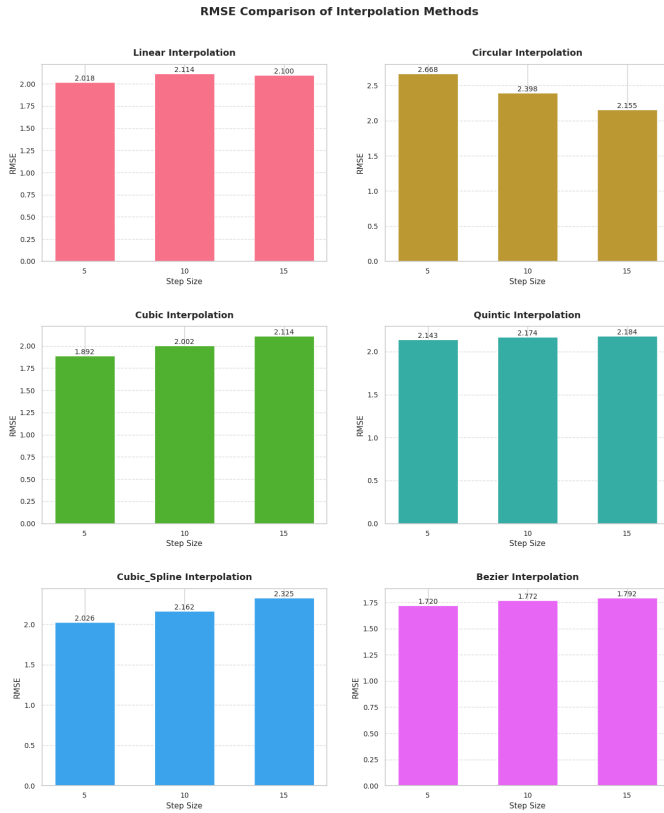


Fig.4 RMSE Comparison of Different Interpolations

C. Implementation Considerations

The ROS2 implementation highlighted practical aspects in terms of execution efficiency and motion smoothness:

- Simpler methods (Linear, Circular) showed moderate performance with RMSE values ranging from 2.0 to 2.7
- Complex methods (Bézier, Quintic) demonstrated better accuracy with RMSE values between 1.7 and 2.2
- Cubic Spline methods showed varying performance based on waypoint density (2.026 to 2.325 RMSE)

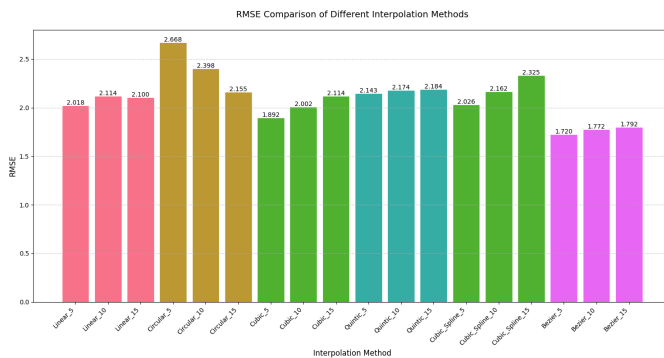


Fig.5 RMSE Comparison

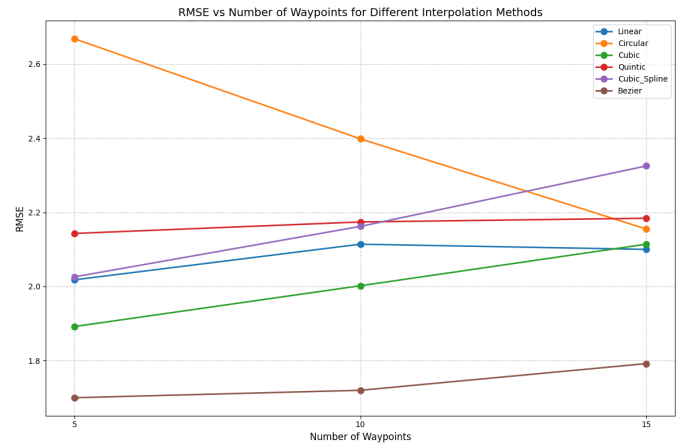


Fig.6 RMSE vs Waypoints

VI. CONCLUSION

This study seeks to understand the performance of organisations when implementing a single supplier concept. several interpolation techniques for trajectory planning in a 7-DoF Panda robot. Our key findings demonstrate that:

- Simpler methods (Linear, Circular) showed moderate performance with RMSE values ranging from 2.0 to 2.7
- Complex methods (Bézier, Quintic) demonstrated better accuracy with RMSE values between 1.7 and 2.2
- Cubic Spline methods showed varying performance based on waypoint density (2.026 to 2.325 RMSE)

These findings extend the existing body of knowledge in robotic motion planning by making it easier to compare interpolation techniques side by side and give practical tips on which method is best suited for which task. Possible future work can include examination of dynamic obstacle avoidance, multirobot operations, optimizing interpolation choice and improving the trajectory based on the type of work required.

REFERENCES

- [1] G. Sotirchos and Z. Ajanovic, "Search-based versus Sampling-based Robot Motion Planning: A Comparative Study," arXiv preprint arXiv:2406.09623, 2024.
- [2] Chengren Yuan. (2024). An efficient RRT cache method in dynamic environments for path planning — Semantic Scholar. <https://www.semanticscholar.org/paper/An-efficient-RRT-cache-method-in-dynamic-for-path-Yuan-Liu/509bbe5b7e9446887acc02f20eef90a8cc39b8e3>
- [3] Y. Liu, Z. Du, M. Liu, Y. Tao and C. Li, "Trajectory Planning of Robot End-Effector Position and Orientation Based on Cubic Spline," 2023 WRC Symposium on Advanced Robotics and Automation (WRC SARA), Beijing, China, 2023, pp. 383-389, doi: 10.1109/WRC-SARA60131.2023.10261781.

- [4] W. Chen, C. Sun, H. Liu, J. Liu and Y. Tang, "Path planning scheme for spray painting robot with Bézier curves on complex curved surfaces," 2017 32nd Youth Academic Annual Conference of Chinese Association of Automation (YAC), Hefei, China, 2017, pp. 698-703, doi: 10.1109/YAC.2017.7967499.
- [5] T. Xu, H. -Z. Liu, R. Huang, Z. Yao and J. -Y. Xu, "Research on Motion Trajectory Planning of Industrial Robot Based on ROS," 2022 7th International Conference on Automation, Control and Robotics Engineering (CACRE), Xi'an, China, 2022, pp. 58-62, doi: 10.1109/CACRE54574.2022.9834167.
- [6] J. He, J. Zhang, J. Liu and X. Fu, "A ROS2-Based Framework for Industrial Automation Systems," 2022 2nd International Conference on Computer, Control and Robotics (ICCCR), Shanghai, China, 2022, pp. 98-102, doi: 10.1109/ICCCR54399.2022.9790247.
- [7] P. Daubaris, A. -P. Tuovinen, S. Linkola, C. Vicente-Chicote, T. Mikkonen and N. Mäkitalo, "ROS2 as an Enabler for a Business Ecosystem: Industrial Collaboration Case Study," 2024 Eighth IEEE International Conference on Robotic Computing (IRC), Tokyo, Japan, 2024, pp. 72-79, doi: 10.1109/IRC63610.2024.00017.
- [8] S. A. Rafiq et al., "Design of Autonomous Rover for Firefighter Rescue: Integrating Deep Learning with ROS2," 2024 IEEE World AI IoT Congress (AIoT), Seattle, WA, USA, 2024, pp. 421-428, doi: 10.1109/AIoT61789.2024.10578950.
- [9] H. Deng, J. Xiong and Z. Xia, "Mobile manipulation task simulation using ROS with MoveIt," 2017 IEEE International Conference on Real-time Computing and Robotics (RCAR), Okinawa, Japan, 2017, pp. 612-616, doi: 10.1109/RCAR.2017.8311930.